

DRL — Exam Cheat Sheet

Bootstrapping means using an estimated value of the future as part of the learning target, rather than waiting for the full empirical return. A **TD error** is the difference between a bootstrap target and the current estimate, and it measures how “surprised” the learner is by a transition.

1) Introduction to DRL (L1)

Markov property: future state distribution depends only on the current state and action.

Bellman equations: express value recursively as immediate reward + discounted future value.

Bellman optimality: replaces expectation over actions with a maximization to define optimal values.

Optimal policy: maximizes expected long-term return and may trade short-term reward for future gain.

Policy iteration: alternate between policy evaluation (estimate values) and policy improvement (greedy update).

Value iteration: repeatedly apply Bellman optimality updates (evaluation + improvement together). Both are dynamic-programming methods requiring a known model and become impractical in large state spaces.

Model-based vs Model-free

Model-based RL: uses or learns environment dynamics to plan ahead; typically more sample-efficient but sensitive to model errors (model bias).

Model-free RL: learns values or policies directly from experience without explicit dynamics; simpler but usually requires more data.

Monte-Carlo (MC) learning: Learns values from complete episode returns (no bootstrapping). Updates only at episode end. unbiased but high variance and slower than TD. The simplest form of model-free learning.

On-policy: learns the policy used to generate data (includes exploration effects).

Off-policy: learns a target policy from data generated by a different behavior policy.

Importance sampling: corrects distribution mismatch between behavior and target policies via likelihood ratios.

Temporal-Difference (TD) learning

TD updates values using one-step bootstrap targets instead of waiting for full episode returns.

Key idea: learn from the TD error (difference between prediction and improved prediction).

Compared to Monte Carlo: TD learns online and is more sample-efficient, but introduces bias from bootstrapping.

SARSA (on-policy TD control)

SARSA updates using the next action actually chosen by the current behavior policy. It therefore learns the value of the exploratory policy itself (including ϵ -greedy exploration). Key intuition: the agent learns to account for exploration risk, often producing safer or more conservative behavior in stochastic or dangerous environments. Typical update uses the observed transition (s, a, r, s', a') and moves $Q(s, a)$ toward the value of (s', a') .

Q-learning (off-policy TD control)

Q-learning updates toward the value of the best possible next action, regardless of which action was actually taken. This makes it off-policy: behavior can be exploratory while learning a greedy target policy. Key intuition: directly approximates the optimal action-value function Q^* . In tabular settings it converges under sufficient exploration, but with function approximation (e.g., neural networks in DQN) instability can appear due to off-policy learning + bootstrapping.

SARSA vs Q-learning

SARSA learns the value of the behavior policy (safer, exploration-aware). Q-learning learns a greedy optimal policy (more aggressive, faster exploitation).

2) Deep Q-Networks (DQN) (L2)

DQN combines Q-learning with a neural network to approximate $Q(s, a)$, enabling value learning in high-dimensional state spaces (e.g., images). A **function approximator** generalizes across states instead of using a tabular representation.

Training is unstable because the target depends on the same network being updated (**non-stationary targets**), and consecutive samples are highly correlated, violating i.i.d. assumptions of SGD.

Experience replay stores transitions and samples them randomly to break correlation, improve data efficiency, and reduce catastrophic forgetting.

Prioritized replay samples transitions with large TD error more often (higher learning signal) and uses importance weights to reduce sampling bias.

Target network uses a delayed copy of the Q-network to compute targets, making updates more stable and preventing feedback loops.

DQN improvements

Advantage function: measures how much better an action is compared to the average value of the state ($A(s, a) = Q(s, a) - V(s)$).

Double DQN reduces overestimation bias by selecting actions with the online network but evaluating them with the target network: $y_t = r_t + \gamma Q_{\theta^-}(s_{t+1}, \arg \max_a Q_{\theta}(s_{t+1}, a'))$.

Dueling DQN decomposes Q-values into state value and advantage, allowing the network to learn useful state values even when actions have similar effects.

Policy gradient methods

Policy gradients optimize the policy directly rather than learning action-values, making them suitable for continuous or very large action spaces.

REINFORCE uses sampled returns to update the policy; unbiased but high variance because gradients rely on noisy returns.

A **baseline** can be subtracted from returns to reduce variance without changing the expected gradient (must not depend on action).

Using the state-value function as a baseline yields the **advantage**, reinforcing actions that perform better than expected.

Actor-critic methods combine an actor (policy) and a critic (value estimator); the critic learns via TD and provides lower-variance learning signals to the actor.

PPO stabilizes training by clipping policy updates, preventing overly large changes while maintaining exploration.

3) Imitation learning (L3)

Imitation learning learns a policy from expert demonstrations when rewards are hard to design or exploration is unsafe.

Apprenticeship Learning: learns a near-expert policy by matching feature expectations of expert demonstrations, implicitly learning a reward function instead of directly copying actions; more robust than Behavioral Cloning but more complex.

Forward Training learns a *non-stationary* policy by training a separate policy for each time step using expert labels on the states actually visited. This reduces compounding errors but becomes expensive for long horizons.

Behavioral cloning treats imitation as supervised learning on expert state-action pairs, but suffers from **covariate shift**.

Compounding error occurs when small mistakes move the agent into unseen states, causing errors to accumulate over time.

Dagger fixes covariate shift by collecting expert labels on states visited by the learner, aligning training with the learner's state distribution.

Dagger with coaching restricts expert actions to ones the learner can realistically imitate, improving stability and learning progression.

4) Multi-armed bandits (L4)

Multi-armed bandits are a simplified RL setting with no state transitions: each action (arm) has an unknown reward distribution, and the goal is to maximize cumulative reward.

Regret is the cumulative loss compared to always selecting the best arm; good algorithms balance exploration and exploitation to keep regret low.

ϵ -greedy chooses a random action with probability ϵ and otherwise exploits the current best estimate; simple and robust but exploration is uninformed and may waste samples.

UCB (Upper Confidence Bound) selects actions using optimistic estimates (mean reward + uncertainty bonus), naturally decreasing exploration as confidence improves.

Gradient Bandits learn a softmax policy by updating action preferences via gradient ascent on reward (often with a baseline), directly optimizing action probabilities rather than estimating values.

Thompson sampling maintains a probabilistic belief over arm quality, samples a model from the posterior, and acts greedily under that sample; exploration emerges naturally from uncertainty.

Contextual bandit extends bandits by conditioning actions on context (state features), but actions do not affect future contexts, so long-term planning and delayed credit assignment are absent.

LinUCB assumes expected reward is linear in context features and computes an optimistic estimate using linear regression with uncertainty bonuses, enabling efficient exploration in high-dimensional contexts.

5) MCTS, AlphaGo/Zero, and AutoML variants (L5)

MCTS (Monte-Carlo Tree Search) is a planning algorithm that incrementally builds a search tree using simulations, balancing exploration and exploitation, and can return the best current action at any time (anytime planning).

The four MCTS stages are **selection** (follow tree policy to a leaf), **expansion** (add new child nodes), **simulation** (rollout to estimate outcome), and **backpropagation** (propagate rewards upward to update node statistics).

A **rollout policy** is a fast approximate policy used during simulation to estimate long-term returns from leaf states.

AlphaGo combines supervised pretraining on expert games with reinforcement learning and MCTS. A policy network narrows the search to promising moves, while a value network evaluates board positions, greatly reducing rollout cost.

AlphaGo Zero removes human data and learns purely via self-play. A single policy-value network both proposes actions and evaluates states, and MCTS uses this network to iteratively generate stronger training targets.

Leaf nodes are evaluated by the learned value network instead of long rollouts, making search faster, more stable, and increasingly stronger through self-play improvement.

AutoML variants

AlphaD3M applies AlphaZero-style self-play and tree search to automatic ML pipeline construction, treating each partial pipeline as a state and pipeline decisions as actions.

A **hierarchical-step plugin** restricts available actions to valid pipeline operations at each stage, reducing search complexity and avoiding invalid configurations.

Neural Architecture Search (NAS) uses reinforcement learning (or related search methods) to automatically discover neural architectures by optimizing validation performance as the reward signal.

DeepLine applies deep RL to construct ML pipelines using a **hierarchical decision process**, where high-level actions choose pipeline stages and low-level decisions select components/hyperparameters, while **action filtering** restricts valid next steps to improve search efficiency and scalability.

6) Meta-learning and transfer learning (L6)

Meta-learning learns how to learn across tasks, optimizing for fast adaptation rather than performance on a single task.

Memory-augmented meta-learning: enables fast one/few-shot adaptation by separating slow parameter learning from fast task-specific adaptation via external memory with read/write controllers; label shuffling forces true memory usage instead of memorization.

SNAIL (Simple Neural Attentive Meta-Learner): processes full episodes as sequences using temporal (dilated) convolutions for local structure and attention for long-range dependencies, enabling few-shot learning without gradient updates at test time.

Model-Agnostic Meta-Learning (MAML): learns a parameter initialization that can adapt to new tasks with only a few gradient steps, explicitly optimizing for fast post-adaptation performance; applicable to supervised and RL settings.

Meta-reinforcement learning (Meta-RL): treats each environment as a task and learns agents that adapt online using recent trajectories, often by encoding task information in hidden states.

ReBAL (Bayesian Meta-RL): a model-based Meta-RL approach maintaining a belief distribution over environment dynamics, enabling uncertainty-aware planning and fast adaptation.

Transfer learning: reuses knowledge learned on previous tasks (e.g., fine-tuning features or policies) to accelerate learning on new tasks, but may cause catastrophic forgetting.

Forward transfer (single-task $\rightarrow$ single-task): train on a source task and reuse learned representations or policies to improve learning speed on a target task.

Energy-Based Policies (EBP): represent policies as energy functions over actions, enabling stochastic sampling, exploration, and connections to maximum-entropy RL.

Progressive networks: prevent forgetting by freezing previously learned networks and adding new columns for new tasks with lateral connections, trading memory growth for stability.

Multi-task transfer (many tasks $\rightarrow$ new task): learn shared representations across many tasks and reuse them to improve generalization and sample efficiency on unseen tasks.

Actor-Mimic networks: train a single multi-task policy by imitating multiple expert policies via matching experts' Q -values (and optionally feature representations), encouraging shared transferable features.

Knowledge distillation: compresses knowledge from a teacher model into a student using soft targets that preserve richer information than hard labels.

Self-supervision for RL: auxiliary objectives (e.g., forward/inverse dynamics or reward prediction) improve representation learning when external rewards are sparse or delayed.

Modular Neural Networks decompose policies into task-specific and robot-specific modules, enabling recombination and transfer across tasks or embodiments.

7) Large state and action spaces (L7)

A **large state space** requires function approximation since tabular methods are infeasible; combining approximation with bootstrapping and off-policy learning can cause instability.

A **large action space** makes exhaustive search and exploration expensive, motivating action embeddings, pruning, factorization, or hierarchical decomposition.

Wolpertinger policy: embeds discrete actions into a continuous space, outputs a proto-action with an actor, retrieves K nearest discrete actions (KNN), and selects the best using a critic (actor-critic + nearest-neighbor search).

Generative Proto-Sequence (GPS): learns a generative model (often a VAE) that proposes entire variable-length discrete action sequences in one shot; a critic evaluates candidate sequences and selects the one with highest predicted return. **Action elimination:** uses (contextual) bandits or confidence estimates to prune unlikely actions early, reducing search while trying to avoid eliminating optimal actions.

Hierarchical RL (h-DQN / FuN): decomposes control into levels: a high-level policy sets goals or sub-tasks (intrinsic rewards), while a low-level controller learns skills to achieve them, improving exploration under sparse rewards.

Feudal Networks (FuN): a manager outputs latent goals every c steps; a worker receives intrinsic reward based on cosine similarity between state change and the goal direction.

Branching Dueling Q-Networks (BDQ): factorize high-dimensional action spaces into independent action branches with a shared state-value backbone, reducing combinatorial complexity.

Joint state-action embeddings: learn shared continuous representations of states and actions to generalize across similar actions and reduce effective action dimensionality.

Value Iteration Network (VIN): neural architecture with an internal differentiable value-iteration module, allowing planning-like computation inside a neural network.

8) Model learning, exploration, and curriculum learning (L8)

Latent-state learning: learn a compact low-dimensional representation where prediction, planning, and control are easier than in raw observation space.

Autoencoder: learns an encoder/decoder that reconstructs observation o from latent z ; compresses data but may keep visually irrelevant features.

VAE: stochastic generative model that reconstructs observations while regularizing the latent distribution (KL term), improving smoothness, sampling, and generalization.

Spatial softmax: converts feature maps into keypoint-like coordinates, preserving spatial "where" information important for visuomotor control.

E2C: jointly learns latent representations and locally linear latent dynamics, enabling trajectory optimization and planning directly in latent space.

Action-conditional video prediction: predicts future observations conditioned on actions; long-horizon prediction suffers compounding error, mitigated using curriculum learning (gradually increase prediction horizon K).

Informed exploration: selects actions expected to reach novel or uncertain states instead of random exploration.

Curiosity / ICM: uses intrinsic reward from forward-model prediction error in learned feature space; an inverse model filters out uncontrollable

environment noise ("noisy TV" problem).

EX²: estimates novelty using exemplar classifiers; states that are easier to discriminate from past data are considered more novel.

Model-based RL: learns a predictive model and performs planning or imagination-based learning inside it, improving sample efficiency but risking **model bias** when the learned model is inaccurate.

9) Transformers in DRL (L9)

Attention: allows each decision step to selectively weight relevant past states/actions, enabling credit assignment over long horizons without recurrence. **Multi-head attention:** multiple attention heads learn different dependency patterns in parallel (e.g., short-term vs long-term structure).

Positional encoding: injects temporal order information since Transformers are permutation-invariant by default.

GTrXL (Gated Transformer-XL): RL-oriented Transformer adding gating (GRU-like residual control) and stable normalization to improve training on long, non-stationary trajectories.

Long-sequence transformers: reduce quadratic attention cost using sparse/structured attention (e.g., sliding windows + global tokens), enabling long-history decision making.

Trajectory Transformer: models RL as sequence modeling over (s, a, r) tokens; the learned model predicts future trajectories, and planning is done via search/optimization over generated sequences.

Decision Transformer: treats offline RL as supervised sequence modeling by conditioning on desired return-to-go and predicting actions autoregressively, avoiding explicit value-function or policy optimization.

Advanced DT Variants

EDT: uses adaptive history length (short history for recovery, long history for stability) and predicts maximal achievable return to improve exploration of better trajectories.

ODT: extends DT to online fine-tuning using entropy regularization, enabling continued exploration and preventing policy collapse during online updates.

CDT: cost-conditioned DT for safe RL; conditions on cost threshold C together with $\{R, s, a\}$, enabling zero-shot adaptation to different safety constraints.

Multi-Game DT: trains a single DT across many tasks using conditioning tokens (task ID or return), enabling shared representations and cross-task generalization.

FDT: improves offline learning using oracle feedback to emphasize high-impact states (large reward variance), focusing learning on critical decision points.

10) Model-based approaches and gradient-free learning (L10)

Model-based RL: learns a dynamics model $\hat{P}$ (and reward) and plans inside it; highly sample-efficient but suffers from **model bias** when the learned model is inaccurate in unseen regions.

Learning the model: four versions (data collection loop)

V1 random data $\rightarrow$ fit model $\rightarrow$ plan; simple but suffers from distribution mismatch between training and policy states.

V2 iterate plan/collect/retrain (Dagger-like), improving data distribution, but open-loop plans are fragile in stochastic environments.

V3 (MPC) replan at every step using the latest state, improving robustness at the cost of high computation at decision time.

V4 differentiate through the learned model to optimize a policy directly; efficient at test time but can be numerically unstable due to compounding model errors.

MPC / CEM: sample candidate action sequences, keep elite samples, and update the sampling distribution toward elites (iterative trajectory optimization).

PILCO: uses Gaussian Process dynamics models to propagate uncertainty during planning, enabling very data-efficient policy search.

LQR: assumes linear dynamics $x_{t+1} = Ax_t + Bu_t$ and quadratic cost, yielding an optimal linear feedback controller $u = -Kx$.

Gradient-free optimization (EA/ES): optimize policy parameters by sampling perturbations and updating in directions correlated with higher returns, avoiding backpropagation through the environment.

Model-free reinforcement learning: learns policies or value functions directly from interaction without explicitly modeling dynamics. It avoids

model bias but is typically less sample-efficient since learning relies on real experience. Examples: Q-learning and DQN (value-based), REINFORCE and actor-critic (policy-based).

11) World models in deep RL (L11)

A **world model** learns a latent representation and latent dynamics so an agent can plan or learn using imagined rollouts instead of only real interaction.

Latent dynamics are preferred over pixel prediction because RL mainly needs decision-relevant structure; latent prediction is lower-dimensional, cheaper, and typically more stable.

Imagination-based learning trains policies on model-generated rollouts, improving sample efficiency but requiring control of compounding model errors.

A typical world model contains an **encoder** (compress observations), a **latent dynamics model** (predict latent transitions given actions), and a **behavior module** (actor-critic or planner operating in latent space).

World model design can be described along four dimensions: (1) **latent representation**, (2) **dynamics formulation**, (3) **planning/behavior strategy**, and (4) **training objective**.

(1) Latent representation type.

Continuous latents (VAE-style) provide smooth optimization and uncertainty regularization but may suffer long-horizon drift.

Discrete latents (VQ-style) provide strong compression and stable imagination but introduce quantization error and biased gradient approximations.

Hybrid latents combine continuous and discrete benefits to balance stability and expressiveness.

RSSL (Representation Self-Supervised Learning).

Uses self-supervised objectives (e.g., contrastive learning or action prediction) to learn task-relevant representations without relying only on reward or pixel reconstruction.

(2) Dynamics formulation.

One-step models predict next latent state; **multi-step models** enforce long-horizon consistency to reduce compounding error.

Latent dynamics (PlaNet, Dreamer) are easier to optimize than **pixel dynamics**, which are high-dimensional and expensive.

(3) Planning and behavior strategy.

Online planning (MPC) optimizes a short action sequence, executes the first action, and replans at every step (robust but computationally expensive).

Latent imagination trains actor-critic policies using imagined rollouts and backpropagation through learned dynamics (sample efficient but sensitive to model errors).

Hybrid strategies combine imagination learning with continual real-data updates to reduce model bias.

(4) Training objectives.

Maximum-likelihood / reconstruction: simple to optimize but may waste capacity on visually irrelevant details.

Task-aligned objectives: optimize prediction quality relevant for control instead of pixels.

Uncertainty minimization: uses ensembles or variance estimates to guide exploration toward uncertain regions.

Contrastive/self-supervised objectives: learn useful dynamics representations without full reconstruction.

PlaNet learns latent dynamics and performs online MPC planning directly in latent space.

PlaNet steps: (1) collect real trajectories, (2) train latent dynamics model with variational objective, (3) plan action sequences using MPC/CEM in latent space, (4) execute first action and repeat planning each step.

PlaNet introduces **latent overshooting**, a multi-step KL objective enforcing long-horizon latent consistency without decoding images.

Dreamer learns a world model and trains an actor-critic entirely inside imagination (no online planning at test time).

Dreamer steps: (1) collect real data, (2) train world model, (3) freeze model, (4) generate imagined rollouts, (5) train actor and critic via backprop through imagined trajectories, (6) collect new real data and repeat.

Dreamer achieves high sample efficiency by replacing MPC with learned policies but depends on model accuracy to avoid learning artifacts.